tidy data with tidyr

prof. dr. nicolas meseth

1 what tidy means
2 wide to long
3 long to wide
4 one cell, many things
5 the map

in a tidy workshop every tool has its own place, so you find it without searching, and tidy data gives every value a place of its own in the same way.



image: ai-generated (gpt-image-2.5-flare)

part 1

what tidy means

one answer, four tables

one row per answer

respondent	brand	price
1	weihen	1.49
1	gut	1.09
1	alpro	1.89
16	weihen	1.19
16	gut	1.19
16	alpro	1.49
36	weihen	1.09
36	gut	1.09
36	alpro	1.49

one answer, four tables

one row per answer

respondent	brand	price
1	weihen	1.49
1	gut	1.09
1	alpro	1.89
16	weihen	1.19
16	gut	1.19
16	alpro	1.49
36	weihen	1.09
36	gut	1.09
36	alpro	1.49

one column per brand

respondent	weihen	gut	alpro
1	1.49	1.09	1.89
16	1.19	1.19	1.49
36	1.09	1.09	1.49

one answer, four tables

one row per answer

respondent	brand	price
1	weihen	1.49
1	gut	1.09
1	alpro	1.89
16	weihen	1.19
16	gut	1.19
16	alpro	1.49
36	weihen	1.09
36	gut	1.09
36	alpro	1.49

one column per brand

respondent	weihen	gut	alpro
1	1.49	1.09	1.89
16	1.19	1.19	1.49
36	1.09	1.09	1.49

all prices in one cell

respondent	prices
1	weihen 1.49, gut 1.09, alpro 1.89
16	weihen 1.19, gut 1.19, alpro 1.49
36	weihen 1.09, gut 1.09, alpro 1.49

one answer, four tables

one row per answer

respondent	brand	price
1	weihen	1.49
1	gut	1.09
1	alpro	1.89
16	weihen	1.19
16	gut	1.19
16	alpro	1.49
36	weihen	1.09
36	gut	1.09
36	alpro	1.49

one column per brand

respondent	weihen	gut	alpro
1	1.49	1.09	1.89
16	1.19	1.19	1.49
36	1.09	1.09	1.49

all prices in one cell

respondent	prices
1	weihen 1.49, gut 1.09, alpro 1.89
16	weihen 1.19, gut 1.19, alpro 1.49
36	weihen 1.09, gut 1.09, alpro 1.49

one column per person

brand	1	16	36
weihen	1.49	1.19	1.09
gut	1.09	1.19	1.09
alpro	1.89	1.49	1.49

one answer, four tables

one row per answer

respondent	brand	price
1	weihen	1.49
1	gut	1.09
1	alpro	1.89
16	weihen	1.19
16	gut	1.19
16	alpro	1.49
36	weihen	1.09
36	gut	1.09
36	alpro	1.49



one column per brand

respondent	weihen	gut	alpro
1	1.49	1.09	1.89
16	1.19	1.19	1.49
36	1.09	1.09	1.49

all prices in one cell

respondent	prices
1	weihen 1.49, gut 1.09, alpro 1.89
16	weihen 1.19, gut 1.19, alpro 1.49
36	weihen 1.09, gut 1.09, alpro 1.49

one column per person

brand	1	16	36
weihen	1.49	1.19	1.09
gut	1.09	1.19	1.09
alpro	1.89	1.49	1.49

all four hold the same numbers, but only the first lets you ask a question with the verbs you know.

respondent	brand	price
1	weihen	1.49
1	gut	1.09
1	alpro	1.89
16	weihen	1.19
16	gut	1.19
16	alpro	1.49
36	weihen	1.09
36	gut	1.09
36	alpro	1.49

respondent	brand	price
1	weihen	1.49
1	gut	1.09
1	alpro	1.89
16	weihen	1.19
16	gut	1.19
16	alpro	1.49
36	weihen	1.09
36	gut	1.09
36	alpro	1.49

every variable has its own column

respondent	brand	price
1	weihen	1.49
1	gut	1.09
1	alpro	1.89
16	weihen	1.19
16	gut	1.19
16	alpro	1.49
36	weihen	1.09
36	gut	1.09
36	alpro	1.49

every observation has its own row

respondent	brand	price
1	weihen	1.49
1	gut	1.09
1	alpro	1.89
16	weihen	1.19
16	gut	1.19
16	alpro	1.49
36	weihen	1.09
36	gut	1.09
36	alpro	1.49

every value has its own cell

what is one observation?

respondent	age	weihen	gut	alpro
1	55	1.49	1.09	1.89
16	46	1.19	1.19	1.49
36	53	1.09	1.09	1.49

respondent	brand	price
1	weihen	1.49
1	gut	1.09
1	alpro	1.89
16	weihen	1.19
16	gut	1.19
16	alpro	1.49
36	weihen	1.09
36	gut	1.09
36	alpro	1.49

what is one observation?

how old are the people who answered?

respondent	age	weihen	gut	alpro
1	55	1.49	1.09	1.89
16	46	1.19	1.19	1.49
36	53	1.09	1.09	1.49



respondent	brand	price
1	weihen	1.49
1	gut	1.09
1	alpro	1.89
16	weihen	1.19
16	gut	1.19
16	alpro	1.49
36	weihen	1.09
36	gut	1.09
36	alpro	1.49

what is one observation?

how old are the people who answered?

respondent	age	weihen	gut	alpro
1	55	1.49	1.09	1.89
16	46	1.19	1.19	1.49
36	53	1.09	1.09	1.49



which brand is thought to be cheapest?

respondent	brand	price
1	weihen	1.49
1	gut	1.09
1	alpro	1.89
16	weihen	1.19
16	gut	1.19
16	alpro	1.49
36	weihen	1.09
36	gut	1.09
36	alpro	1.49



what counts as one observation depends on the question, so tidy is a property of a table and a question together.

why it pays off

one column per brand

```
prices |>
  summarize(
    weihen = median(weihen),
    gut = median(gut),
    alpro = median(alpro)
  )
```

one line for every brand

one row per person and brand

```
prices_long |>
  group_by(brand) |>
  summarize(median = median(price))
```

one line for all brands

in a tidy table one line of code covers every brand, however many there are.

respondents gave a price for each of three brands, one column per brand. is this table tidy?

yes, one row per person

no, values in column names

no, two values in one cell

it depends on the question

live.hsos.uber.space/q

respondents gave a price for each of three brands, one column per brand. is this table tidy?

- yes, one row per person
- no, values in column names
- no, two values in one cell
- it depends on the question

live.hsos.uber.space/q

respondents gave a price for each of three brands, one column per brand. is this table tidy?

- yes, one row per person
- no, values in column names
- no, two values in one cell
- it depends on the question

live.hsos.uber.space/q

four ways to be messy

values in column names

respondent	weihen	gut	alpro
1	1.49	1.09	1.89
16	1.19	1.19	1.49

`pivot_longer()`

two variables in one column

respondent	measure	value
1	price	1.49
1	sure	2
16	price	1.19

`pivot_wider()`

two values in one cell

brand	range
weihen	1.30 to 1.50
gut	0.90 to 1.10

`separate_wider_delim()`

a list in one cell

respondent	shops
12	supermarket
24	discounter, supermarket, farm shop

`separate_longer_delim()`

part 2

wide to long

```
library(tidyverse)

raw <- read_csv(
  "https://hands-on-computer-science.de/media/datasets/mds12_schoko_milch.csv"
) |>
  mutate(respondent = row_number(), .before = 1)
```

one row per respondent, 2,811 rows and 606 columns, but no column that tells the respondents apart, so `row_number()` gives every row a number in the new column `respondent`

the price question, as the survey stores it

respondent	u013pzahl_1weihen	u013pzahl_2gut	u013pzahl_3alpro
1	1.49	1.09	1.89
2	NA	NA	NA
3	NA	NA	NA
…	…	…	…
16	1.19	1.19	1.49
…	…	…	…
36	1.09	1.09	1.49

2,580 of the 2,811 rows are empty in all three columns,
because only 231 people were asked this question

the brand is a variable, but here it hides in three column names.

pivot_longer(): columns become rows

respondent	weihen	gut	alpro
1	1.49	1.09	1.89
16	1.19	1.19	1.49
36	1.09	1.09	1.49

pivot_longer(): columns become rows

respondent	weihen	gut	alpro
1	1.49	1.09	1.89
16	1.19	1.19	1.49
36	1.09	1.09	1.49

pivot_longer(): columns become rows

respondent	weihen	gut	alpro
1	1.49	1.09	1.89
16	1.19	1.19	1.49
36	1.09	1.09	1.49

respondent	brand	price
1	weihen	1.49
1	gut	1.09
1	alpro	1.89

pivot_longer(): columns become rows

respondent	weihen	gut	alpro
1	1.49	1.09	1.89
16	1.19	1.19	1.49
36	1.09	1.09	1.49

respondent	brand	price
1	weihen	1.49
1	gut	1.09
1	alpro	1.89
16	weihen	1.19
16	gut	1.19
16	alpro	1.49
36	weihen	1.09
36	gut	1.09
36	alpro	1.49

pivot_longer(): columns become rows

	names_to	values_to
respondent	brand	price
1	weihen	1.49
1	gut	1.09
1	alpro	1.89
16	weihen	1.19
16	gut	1.19
16	alpro	1.49
36	weihen	1.09
36	gut	1.09
36	alpro	1.49

```
prices <- tribble(
  ~respondent, ~weihen, ~gut, ~alpro,
  1,           1.49,    1.09, 1.89,
  16,          1.19,    1.19, 1.49,
  36,          1.09,    1.09, 1.49
)

prices |>
  pivot_longer(
    cols = c(weihen, gut, alpro),
    names_to = "brand",
    values_to = "price"
  )
```

which columns?
where do their names go?
where do their values go?

with the real survey

```
raw |>
  select(respondent, age = Q002alter, starts_with("u013pzahl")) |>
  pivot_longer(
    cols = starts_with("u013pzahl"),
    names_to = "brand",
    values_to = "price"
  )
```

with the real survey

```
raw |>
  select(respondent, age = Q002alter, starts_with("u013pzahl")) |>
  pivot_longer(
    cols = starts_with("u013pzahl"),
    names_to = "brand",
    values_to = "price"
  )
```

```
# A tibble: 8,433 × 4
```

respondent	age	brand	price
1	55	u013pzahl_1weihen	1.49
1	55	u013pzahl_2gut	1.09
1	55	u013pzahl_3alpro	1.89
2	35	u013pzahl_1weihen	NA
2	35	u013pzahl_2gut	NA
2	35	u013pzahl_3alpro	NA

cleaning up on the way

```
prices_long <- raw |>
  select(respondent, age = Q002alter, starts_with("u013pzahl")) |>
  pivot_longer(
    cols = starts_with("u013pzahl"),
    names_to = "brand",
    names_pattern = "u013pzahl_\\d(.*)",
    values_to = "price",
    values_drop_na = TRUE
  )
```

`names_prefix`	removes a fixed start of the name
`names_pattern`	keeps what the brackets catch
`names_sep`	splits the name at a character
`values_drop_na`	drops the empty values

more arguments of pivot_longer()

cleaning up on the way

```
prices_long <- raw |>
  select(respondent, age = Q002alter, starts_with("u013pzahl")) |>
  pivot_longer(
    cols = starts_with("u013pzahl"),
    names_to = "brand",
    names_pattern = "u013pzahl_\\d(.*)",
    values_to = "price",
    values_drop_na = TRUE
  )
```

`names_prefix`	removes a fixed start of the name
`names_pattern`	keeps what the brackets catch
`names_sep`	splits the name at a character
`values_drop_na`	drops the empty values

more arguments of pivot_longer()

```
# A tibble: 693 × 4
```

respondent	age	brand	price
1	55	weihen	1.49
1	55	gut	1.09
1	55	alpro	1.89

now one line answers it

```
prices_long |>
  filter(price <= 10) |>
  group_by(brand) |>
  summarize(n = n(), median = median(price))
```

now one line answers it

```
prices_long |>
  filter(price <= 10) |>
  group_by(brand) |>
  summarize(n = n(), median = median(price))
```

brand	n	median
alpro	224	1.6
gut	226	1.09
weihen	225	1.49

respondents find a higher price fair for alpro than for weihenstephan, although the survey showed both at 1.79.

nine columns of zeros and ones

respondent	v008ort_1discount	v008ort_2super	v008ort_3sbmarkt	v008ort_4fach
1	1	1	0	0
2	1	1	0	1
3	1	1	0	0
4	1	0	1	0

```
# 5 more variables: v008ort_5markt, v008ort_6hof,
#   v008ort_7bioladen, v008ort_8onlineds, v008ort_9online
```

a question with several answers is one variable, however many columns the survey tool makes of it.

```
raw |>
  select(respondent, starts_with("v008ort")) |>
  pivot_longer(
    cols = starts_with("v008ort"),
    names_to = "shop",
    names_prefix = "v008ort_",
    values_to = "value"
  ) |>
  filter(value == 1) |>
  count(shop)
```

```
raw |>
  select(respondent, starts_with("v008ort")) |>
  pivot_longer(
    cols = starts_with("v008ort"),
    names_to = "shop",
    names_prefix = "v008ort_",
    values_to = "value"
  ) |>
  filter(value == 1) |>
  count(shop)
```

shop	n
1discount	2065
2super	2134
3sbmarkt	848
4fach	1029
5markt	493
6hof	396
7bioladen	359
8onlineds	303
9online	173

part 3

long to wide

respondent	brand	measure	value
1	weihen	price	1.49
1	weihen	sure	2
1	alpro	price	1.89
1	alpro	sure	-1
16	weihen	price	1.19
16	weihen	sure	3
16	alpro	price	1.49
16	alpro	sure	1

price and certainty are two variables, so they need two columns, even if the table looks tidy at first sight.

pivot_wider(): rows become columns

respondent	brand	measure	value
1	weihen	price	1.49
1	weihen	sure	2
1	alpro	price	1.89
1	alpro	sure	-1
16	weihen	price	1.19
16	weihen	sure	3
16	alpro	price	1.49
16	alpro	sure	1

pivot_wider(): rows become columns

respondent	brand	measure	value
1	weihen	price	1.49
1	weihen	sure	2
1	alpro	price	1.89
1	alpro	sure	-1
16	weihen	price	1.19
16	weihen	sure	3
16	alpro	price	1.49
16	alpro	sure	1

pivot_wider(): rows become columns

respondent	brand	measure	value
1	weihen	price	1.49
1	weihen	sure	2
1	alpro	price	1.89
1	alpro	sure	-1
16	weihen	price	1.19
16	weihen	sure	3
16	alpro	price	1.49
16	alpro	sure	1

respondent	brand	price	sure
1	weihen	1.49	2

pivot_wider(): rows become columns

respondent	brand	measure	value
1	weihen	price	1.49
1	weihen	sure	2
1	alpro	price	1.89
1	alpro	sure	-1
16	weihen	price	1.19
16	weihen	sure	3
16	alpro	price	1.49
16	alpro	sure	1

respondent	brand	price	sure
1	weihen	1.49	2
1	alpro	1.89	-1
16	weihen	1.19	3
16	alpro	1.49	1

the code

```
answers <- tribble(
  ~respondent, ~brand,   ~measure, ~value,
  1,           "weihen", "price",  1.49,
  1,           "weihen", "sure",   2,
  1,           "alpro",  "price",  1.89,
  1,           "alpro",  "sure",   -1,
  16,          "weihen", "price",  1.19,
  16,          "weihen", "sure",   3,
  16,          "alpro",  "price",  1.49,
  16,          "alpro",  "sure",   1
)

answers |>
  pivot_wider(
    names_from = measure,
    values_from = value
  )
```

which column gives the new names?
which column fills them?

a long table has 3 people, 3 brands and 2 measures in rows. how many rows does it have?

6

9

18

27

live.hsos.uber.space/q

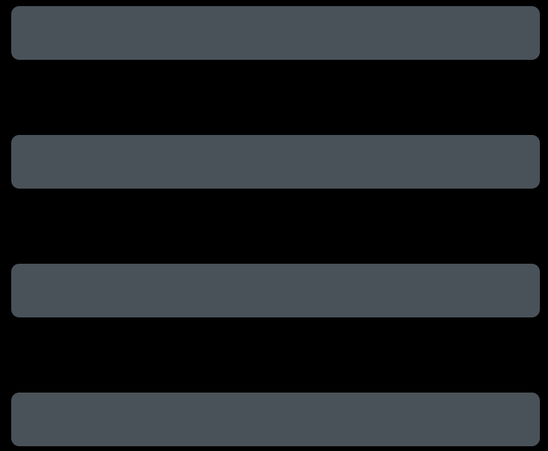
a long table has 3 people, 3 brands and 2 measures in rows. how many rows does it have?

6

9

18

27



live.hsos.uber.space/q

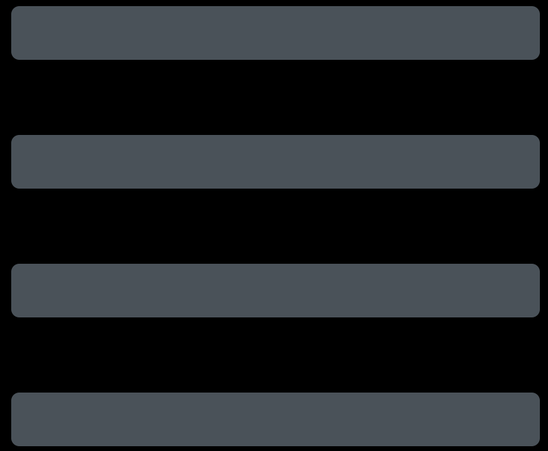
a long table has 3 people, 3 brands and 2 measures in rows. how many rows does it have?

6

9

18

27



live.hsos.uber.space/q

```
prices_long |>
  filter(price <= 10) |>
  mutate(decade = age %/% 10 * 10) |>
  group_by(decade, brand) |>
  summarize(median = median(price)) |>
  pivot_wider(
    names_from = brand,
    values_from = median
  )
```

keep data long while you analyse it, and make it wide only when a person has to read it.

```
prices_long |>
  filter(price <= 10) |>
  mutate(decade = age %/% 10 * 10) |>
  group_by(decade, brand) |>
  summarize(median = median(price)) |>
  pivot_wider(
    names_from = brand,
    values_from = median
  )
```

decade	alpro	gut	weihen
10	1.80	1.50	1.80
20	1.80	1.10	1.80
30	1.69	1.00	1.40
40	1.50	1.09	1.35
50	1.60	1.09	1.50
60	1.59	1.09	1.39
70	1.40	1.11	1.20

keep data long while you analyse it, and make it wide only when a person has to read it.

respondent	weihen	gut	alpro
1	1.49	1.09	1.89
16	1.19	1.19	1.49
36	1.09	1.09	1.49

pivot_longer()

pivot_wider()

respondent	brand	price
1	weihen	1.49
1	gut	1.09
1	alpro	1.89
16	weihen	1.19
16	gut	1.19
16	alpro	1.49
36	weihen	1.09
36	gut	1.09
36	alpro	1.49

two variables across two brands

respondent	price_weihen	price_alpro	sure_weihen	sure_alpro
1	1.49	1.89	2	-1
16	1.19	1.49	3	1

two variables across two brands

respondent	price weihen	price alpro	sure weihen	sure alpro
1	1.49	1.89	2	-1
16	1.19	1.49	3	1

two variables across two brands

respondent	brand	price	sure
1	weihen	1.49	2
1	alpro	1.89	-1
16	weihen	1.19	3
16	alpro	1.49	1

names_to with ".value"

```
both <- tribble(
  ~respondent, ~price_weihen, ~price_alpro, ~sure_weihen, ~sure_alpro,
  1,           1.49,          1.89,         2,            -1,
  16,          1.19,          1.49,         3,            1
)

both |>
  pivot_longer(
    cols = -respondent,
    names_to = c(".value", "brand"),
    names_sep = "_"
  )
```

the result is the tidy table with the columns `respondent`, `brand`, `price` and `sure`

part 4

one cell, many things

brand	range
weihen	1.30 to 1.50
gut	0.90 to 1.10
alpro	1.50 to 1.70

brand	range
weihen	1.30 to 1.50
gut	0.90 to 1.10
alpro	1.50 to 1.70

brand	from	to
weihen	1.30	1.50
gut	0.90	1.10
alpro	1.50	1.70

```
ranges <- tribble(
  ~brand,   ~range,
  "weihen", "1.30 to 1.50",
  "gut",    "0.90 to 1.10",
  "alpro",  "1.50 to 1.70"
)

ranges |>
  separate_wider_delim(
    range,
    delim = " to ",
    names = c("from", "to")
  ) |>
  mutate(from = as.numeric(from), to = as.numeric(to))
```

a list in one cell

respondent	shops
12	supermarket
24	discounter, supermarket, farm shop
48	supermarket, specialist shop, organic shop

a list in one cell

respondent	shops
12	supermarket
24	discounter, supermarket, farm shop
48	supermarket, specialist shop, organic shop

a list in one cell

respondent	shops
12	supermarket
24	discounter
24	supermarket
24	farm shop
48	supermarket
48	specialist shop
48	organic shop

```
shopping <- tribble(
  ~respondent, ~shops,
  12,          "supermarket",
  24,          "discounter, supermarket, farm shop",
  48,          "supermarket, specialist shop, organic shop"
)

shopping |>
  separate_longer_delim(shops, delim = ", ") |>
  count(shops)
```

wider splits a cell into columns, longer splits it into rows, and the question decides which one you need.

```
shopping <- tribble(
  ~respondent, ~shops,
  12,          "supermarket",
  24,          "discounter, supermarket, farm shop",
  48,          "supermarket, specialist shop, organic shop"
)

shopping |>
  separate_longer_delim(shops, delim = ", ") |>
  count(shops)
```

shops	n
discounter	1
farm shop	1
organic shop	1
specialist shop	1
supermarket	3

wider splits a cell into columns, longer splits it into rows, and the question decides which one you need.

one question, three shapes

a list in one cell

respondent	shops
12	supermarket
24	discounter, supermarket, farm shop
48	supermarket, specialist shop, organic shop



tidy: one row per person and shop

respondent	shop
12	supermarket
24	discounter
24	supermarket
24	farm shop
48	supermarket
48	specialist shop
48	organic shop

one question, three shapes

a list in one cell

respondent	shops
12	supermarket
24	discounter, supermarket, farm shop
48	supermarket, specialist shop, organic shop

`separate_longer_delim()`

one column per shop, 1 means ticked

respondent	discounter	supermarket	specialist shop	farm shop	organic shop	+ 4 more
12	0	1	0	0	0	
24	1	1	0	1	0	
48	0	1	1	0	1	

`pivot_longer() |> filter()`

tidy: one row per person and shop

respondent	shop
12	supermarket
24	discounter
24	supermarket
24	farm shop
48	supermarket
48	specialist shop
48	organic shop

a cell holds "vegan, organic, regional" and you want to count how often each label occurs. which verb?

pivot_longer()

pivot_wider()

separate_wider_delim()

separate_longer_delim()



live.hsos.uber.space/q

a cell holds "vegan, organic, regional" and you want to count how often each label occurs. which verb?

pivot_longer()

pivot_wider()

separate_wider_delim()

separate_longer_delim()

live.hsos.uber.space/q

a cell holds "vegan, organic, regional" and you want to count how often each label occurs. which verb?

pivot_longer()

pivot_wider()

separate_wider_delim()

separate_longer_delim()

live.hsos.uber.space/q

part 5

the map

in a spreadsheet a group is often named only once, at the top of its block, because the eye fills in the rest.

brand	respondent	price
weihen	1	1.49
	16	1.19
	36	1.09
alpro	1	1.89
	16	1.49
	36	1.49

```
sheet <- tribble(
  ~brand,   ~respondent, ~price,
  "weihen", 1,           1.49,
  NA,       16,          1.19,
  NA,       36,          1.09,
  "alpro",  1,           1.89,
  NA,       16,          1.49,
  NA,       36,          1.49
)

sheet |>
  fill(brand)
```

brand	respondent	price
weihen	1	1.49
NA	16	1.19
NA	36	1.09
alpro	1	1.89
NA	16	1.49
NA	36	1.49

```
sheet <- tribble(
  ~brand,   ~respondent, ~price,
  "weihen", 1,           1.49,
  NA,       16,          1.19,
  NA,       36,          1.09,
  "alpro",  1,           1.89,
  NA,       16,          1.49,
  NA,       36,          1.49
)

sheet |>
  fill(brand)
```

brand	respondent	price
weihen	1	1.49
weihen	16	1.19
weihen	36	1.09
alpro	1	1.89
alpro	16	1.49
alpro	36	1.49

from symptom to verb

what you see	what you do
values in column names	`pivot_longer()`
two variables in one column	`pivot_wider()`
two values in one cell	`separate_wider_delim()`
a list in one cell	`separate_longer_delim()`
a group named only once	`fill()`

tidying comes first and
costs time, but after it
every new question is only
a few lines of code.

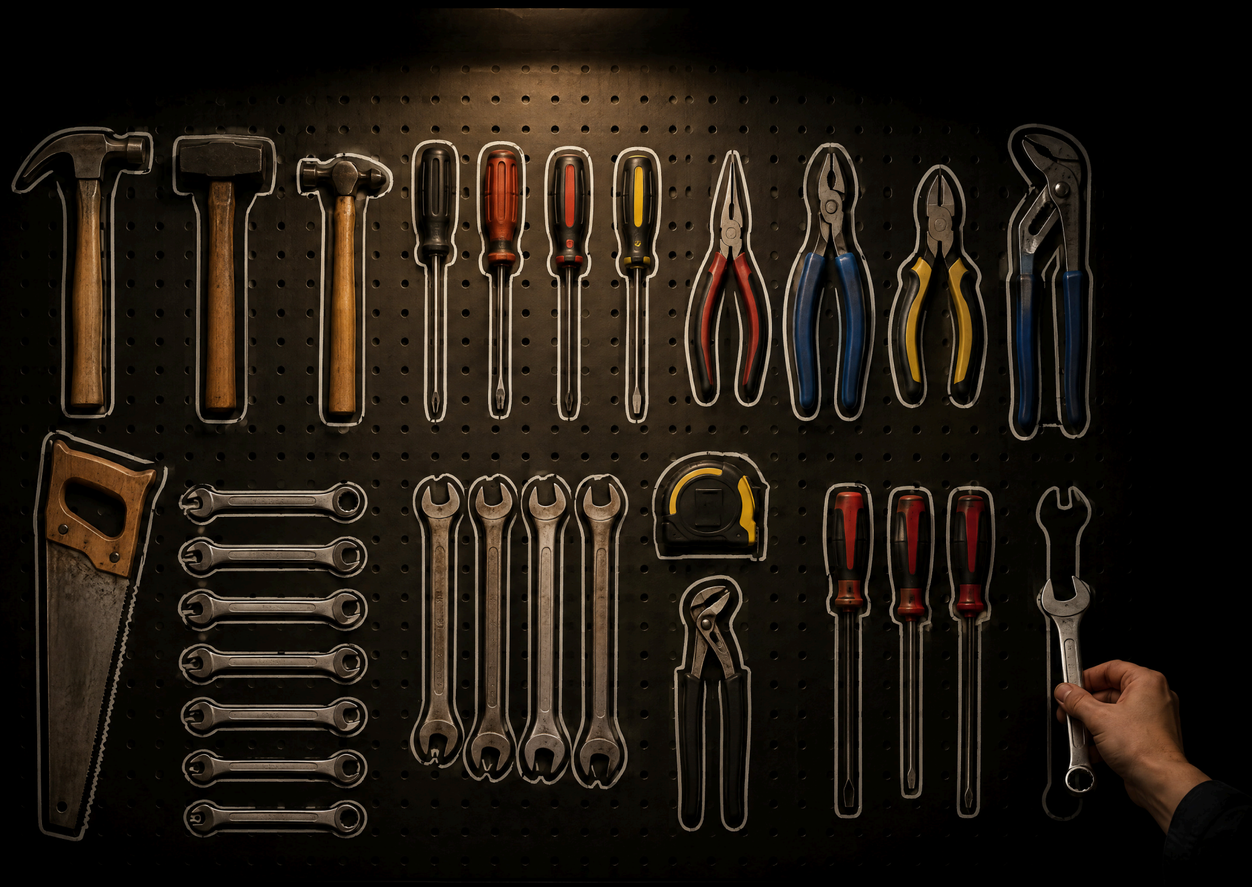


image: ai-generated (gpt-image-2.5-flare)